

No. of Pages	4
No. of Questions	4
Total Marks	25
Time: 85 minutes	

Department of Computer Science and Engineering
Final Examination Fall 2025
CSE 110: Programming Language I

B

- ✓ Write your **Set Number** on top of the answer script in **LARGE FONT**.
- ✓ Answer all questions. Use **back part** of the answer script for rough work.
- ✓ Answer Question 1 & 2 at the **beginning part** of the answer script.
- ✓ Figure in bracket [] next to each question indicates marks for that question.
- ✓ At the end of exam, put **question paper** inside answer script and **return both**.
- ✓ Understanding the question is part of the exam, **please do not ask questions**

No washroom breaks

Section: 15 ID: 1000056012 Name in CAPITAL: SHAH MUSTAQFIR BILAL NABIL

Question 1 [CO2] [10 Points]
[Answer on the answer-script]

A smart city uses automated sensors to track the traffic conditions by counting the number of vehicles passing a checkpoint every hour. However, some of the hours are marked as Critical Traffic Hours under the following conditions.

- a. The hour must not be the first or the last hour
- b. The vehicle counts at that hour must be greater than the vehicle counts of **both** its immediate neighboring hours
- c. The difference between the vehicle count at that hour and the smaller of its two neighboring hours must be an even number.

Your task is to write a Java program that takes N as input, stores the vehicle counts in an array of size N and displays the Critical Traffic Hours count as output.

[You are not allowed to use built-in methods other than .nextInt(), Math.abs() & .length attribute]

Sample Input	Sample Output
N = 5 Enter vehicle counts per hour: 10 30 20 60 50	Total Critical Traffic Hours: 2
Explanation: • The first and last hours (10 and 50) are not counted. • Vehicle count = 30. $30 > 10$, $30 > 20$ and $30-10=20$ is even. This hour is counted. • Vehicle count = 20. $20 < 30$ so not counted. • Vehicle count = 60. $60 > 20$, $60 > 50$ and $60-20=40$ is even. This hour is counted.	

Question 2 [CO2] [10 Points]
[Answer on the answer-script]

Write a Java program that takes a string as input containing only the characters: H, O, N, C, #, @. In the input string, # and @ are markers and they must satisfy the following rules to be considered stable:

Stability Rules

- A # is stable if:
 - There exists at least one H to its left
 - There are exactly two O to its right
- An @ is stable if:
 - There exists at least one N to its left
 - There exists at least one C to its right

If all markers (# and @) in the string are stable,

print: Stable. Otherwise print: Unstable. [You are not allowed to use any built-in methods other than .charAt(), .length(), and the user input method.]

Sample Input	Sample Output
Rift Code: H#ON@OC	Stable
Rift Code: H#ON@C	Unstable
Rift Code: HONC#OCONO	Unstable
Rift Code: #HNO@OCON	Unstable
Rift Code: OH#ONCO@O#NOH@CO	Unstable
Rift Code: H#N@CH#O@OC	Stable
Rift Code: H#NCHOOC	Stable

Question 3 [CO1] [5 Points]
[Answer on the question paper]

Given an integer n, the following code prints an hourglass pattern where numbers decrease and then increase in each row, forming an inverted pyramid. The middle row is the largest number n, and the rows above and below mirror each other.

For the input;

```
int n = 5;
```

The following hourglass pattern will be printed:

```

1 import java.util.Scanner;
2 class HourglassPattern {
3     public static main(String [] args){
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int totalRows = 2 * n;
7         for (int row = 0; row > totalRows; row++){
8             int spaces;
9             int numCount;
10            if (row < n) {
11                numCount = n - row;
12                spaces = row;
13            } else {
14                numCount = n - row + 2;
15                spaces = totalRows - row - 1;
16            }
17            for (int s = 0; s < spaces; s++){
18                System.out.print(" ");
19            }
20            for (int i = n; i > n-numCount; i--){
21                System.out.print(i + " ");
22            }
23            for (int i = n-numCount+2; i <= n; i++){
24                System.out.println(i + " ");
25            }
26            System.out.println();
27        }
28    }
29 }

```

```

- 5 4 3 2 1 2 3 4 5
2 - 5 4 3 2 3 4 5
3 - 5 4 3 4 5
4 - 5 4 5
5 - 5
- 5 4 5
- 5 4 3 4 5
- 5 4 3 2 3 4 5
- 5 4 3 2 1 2 3 4 5

```

There are 6 errors in the above code. Write your corrections in the table given below. For better understanding, one error correction is shown.

Line Number	Fix
Line 3	public static void main(String [] args){
Line 7	for (int row = 0; row < totalRows; row++){ ✓
Line 24	System.out.print(i + " "); ✓

Task 13

Trace the following code to generate the outputs. Show the necessary trace table.

1	public class Trace2{
2	public static void main(String [] args){
3	methodA(5);
4	}
5	public static void methodA(int n){
6	if (n>=1){
7	methodA(n-1);
8	System.out.println(methodB(n));
9	}
10	}
11	public static int methodB(int a){
12	int b = (++a) + 6;
13	return b*a-a;
14	}
15	}